

# **Implantación y monitorización de un sistema web para la gestión de un taller mecánico**

Proyecto final de ciclo — CFGS Administración de Sistemas Informáticos en Red

Autor: Antonio Corredera Cubells

Curso académico: 2025/2026

Centro docente: IES Camp de Morvedre

Tutor/a individual: Hueso Pastor, Francisco Alfonso

*Guion oral completo para defensa de 30 minutos*

## Índice operativo

| Minutos | Contenido                                   | Evidencia                      |
|---------|---------------------------------------------|--------------------------------|
| 0–5     | Contexto, objetivos y módulos ASIR          | Portada, intro, tabla módulos  |
| 5–12    | App: login, citas, noticias, administración | Navegador — flujos principales |
| 12–18   | Docker, BD, volúmenes y redes               | docker compose ps, esquema     |
| 18–24   | Prometheus, Grafana y simulador             | Dashboards y carga sintética   |
| 24–28   | AWS, seguridad y presupuesto                | Diagrama despliegue y costes   |
| 28–30   | Conclusiones, mejoras y preguntas           | Tabla objetivos y cierre       |

### Uso recomendado

- No leer las diapositivas: usar cada una como apoyo visual y el guion como ensayo previo.
- En demos, explicar primero que se va a comprobar y después mostrarlo. Evitar navegar sin narración.
- No proyectar credenciales, tokens, `.env`` completo ni consolas con secretos.
- Si una demo falla, convertirlo en procedimiento operativo: revisar contenedores, targets y logs.
- Mantener la diferencia entre citación (acción de reservar) y cita (registro creado).
- Cerrar cada bloque con una frase puente para que el tribunal vea continuidad.

## Guion por diapositiva

### 01. Implantación y monitorización de un sistema web para la gestión de un taller mecánico

**Tiempo estimado: 0:00-0:30**

*Objetivo comunicativo: Abrir con seguridad, identificar el proyecto y situar el alcance técnico.*

#### **Mi texto**

Buenos días. Soy Antonio Corredera Cubells y voy a presentar mi Proyecto Final de Ciclo: implantación y monitorización de un sistema web para la gestión de un taller mecánico.

#### **Texto oral sugerido**

Buenos días. Soy Antonio Corredera Cubells y voy a presentar mi Proyecto Final de Ciclo: implantación y monitorización de un sistema web para la gestión de un taller mecánico.

El proyecto toma un caso de PYME muy concreto, un taller mecánico, y lo utiliza para demostrar un ciclo completo de administración de sistemas: una aplicación web LAMP, una base de datos relacional, despliegue reproducible con Docker, despliegue documentado en AWS y observabilidad con Prometheus y Grafana.

La idea principal no es solo mostrar una web, sino defender como se prepara un servicio para operar, medirse y mantenerse.

#### **Indicaciones de demo o énfasis**

- No leer la portada. Mirar al tribunal, indicar nombre y título, y avanzar rápido.

*Transición: Empiezo con el mapa de la exposición para que se vea dónde encajan las demos.*

### 02. Mapa de la exposición (30 min) — Tabla 16

**Tiempo estimado: 0:30-1:15**

*Objetivo comunicativo: Explicar la estructura de 30 minutos y anticipar las evidencias.*

#### **Texto oral sugerido**

La defensa está organizada en seis bloques. Primero presentaré el contexto, los objetivos y la relación con los módulos de ASIR. Después pasaré a la aplicación: login, citaciones, noticias y administración.

A continuación defenderé la parte de implantación: Docker Compose, base de datos, volúmenes y redes. El cuarto bloque es la monitorización, con Prometheus, Grafana y el simulador de tráfico.

Finalmente explicare AWS, seguridad, presupuesto, validación y conclusiones. He reservado margen para preguntas, porque varias decisiones técnicas son defendibles desde el punto de vista de operación.

#### **Indicaciones de demo o énfasis**

- Avisar de que habrá demo en navegador, terminal y Grafana.
- No detenerse en cada fila de la tabla; usarla como contrato de tiempo.

*Transición: Con ese recorrido, el primer punto es explicar por qué este caso de taller es útil para un PFC de ASIR.*

### **03. Contexto: digitalización de PYMEs**

**Tiempo estimado: 1:15-2:00**

**Mio:**

El punto de partida es una situación habitual en una pequeña empresa en la que aun se atienden citas por teléfono, se apuntan los datos a papel y lápiz causando pérdida de información o datos duplicados.

Todo esto surge cuando un taller mecánico tiene la necesidad de digitalizar su negocio. Realizaremos una aplicación Web, base de datos, despliegue en aws utilizando contenedores Docker y todo esto lo monitozaremos utilizando Jmeter, prometheus y Grafana.

*Objetivo comunicativo: Conectar el problema de negocio con el alcance ASIR.*

#### **Texto oral sugerido**

El punto de partida es una situación habitual en una pequeña empresa de servicios: citas gestionadas por teléfono, papel o WhatsApp, información duplicada y poca trazabilidad.

Digitalizar aquí no significa crear una página informativa y ya está. Significa ordenar datos, separar roles, controlar el acceso, desplegar de forma repetible y observar si el sistema funciona.

Por eso el taller es un caso acotado: tiene citas, usuarios, administración y contenido público, pero permite demostrar infraestructura, red, seguridad, cloud y métricas sin convertir el proyecto en un producto comercial enorme.

#### **Indicaciones de demo o énfasis**

- Enfatizar la expresión caso\_taller: negocio acotado, alcance ASIR amplio.

*Transición: A partir de ese contexto, los objetivos se pueden medir de forma bastante directa.*

## 04. Objetivos del PFC

**Tiempo estimado: 2:00-3:00**

*Objetivo comunicativo: Presentar objetivos verificables y no solo funcionales.*

**Mio:**

Objetivos del PFC o necesidades para la empresa:

- Diseñar una aplicación web funcional para la gestión del taller: Login, citas y demás
- Crear una base de datos relacional con usuarios, noticias y citas.
- Publicar los contenedores en docker para que el despliegue sea reproducible.
- Incorporar monitorización para saber la salud del sistema.
- Documentar el proceso de despliegue en AWS y costes operativos.

### **Texto oral sugerido**

Los objetivos se agrupan en cinco líneas. La primera es diseñar una aplicación web funcional para la gestión básica del taller: registro, login, citas, noticias, consejos y administración.

La segunda es sostener esa aplicación con una base de datos relacional, con tablas para usuarios, citas y contenido, y con integridad referencial para evitar inconsistencias.

La tercera es publicar el sistema con Docker para que el despliegue sea reproducible en local y en cloud. La cuarta es incorporar monitorización: salud del sistema, uso de la aplicación, base de datos y métricas de negocio.

Y la quinta es documentar una implantación en AWS, incluyendo costes orientativos, seguridad perimetral y tareas de mantenimiento.

### **Indicaciones de demo o énfasis**

- No leer literalmente las cinco viñetas; convertirlas en cinco compromisos verificables.

*Transición: Para cubrir esos objetivos he combinado tecnologías de desarrollo web, sistemas, red y observabilidad.*

## 05. Stack tecnológico del PFC

**Tiempo estimado: 3:00-4:15**

*Objetivo comunicativo: Defender que el proyecto integra competencias del ciclo, no una única asignatura.*

**Mio:**

El stack tecnológico combina un enfoque clásico LAMP con herramientas actuales, para la web Apache y PHP con ayuda de MySQL con la infraestructura de Docker y Docker Compose.

Sobre esa base añadimos Prometheus para recopilar métricas, Grafana para visualizarlas, Alert manager para las alertas y JMeter para generar la carga.

(Podría añadir el último párrafo de TextoOralSugerido pero no lo sé. El que empieza con esta mezcla es importante...)

### **Texto oral sugerido**

El stack tecnológico combina un enfoque clásico LAMP con herramientas actuales de operación. En la capa web se usa Apache y PHP; en persistencia, MySQL; en infraestructura, Docker y Docker Compose.

Sobre esa base se añade Prometheus para recopilar métricas, Grafana para visualizarlas, Alertmanager para alertas y JMeter para generar carga controlada.

Esta mezcla es importante para ASIR porque obliga a trabajar con servicios, puertos, volúmenes, variables de entorno, logs, backups, seguridad y despliegue. La aplicación es el hilo conductor, pero el valor académico está en operar el sistema completo.

### **Indicaciones de demo o énfasis**

- Señalar el gráfico de barras como reparto aproximado de peso técnico.

*Transición: La siguiente diapositiva traduce ese stack a módulos concretos del ciclo.*

## 06. Mapa de competencias ASIR (Tabla 1)

**Tiempo estimado: 4:15-5:00**

*Objetivo comunicativo: Vincular evidencias concretas con módulos ASIR.*

**Mio:**

### **Texto oral sugerido**

Esta tabla resume la correspondencia con los módulos. En Implantación de Aplicaciones Web aparecen PHP, Apache, formularios, sesiones y el panel de administración.

En Bases de Datos y Administración de SGBD se trabaja con MySQL, relaciones, índices, inicialización SQL y usuarios de aplicación. En Sistemas, Redes y Seguridad entran Docker Compose, volúmenes, healthchecks, roles, contraseñas cifradas, CSRF y Security Groups.

También hay documentación técnica en Markdown, YAML y JSON de dashboards. Por tanto, no es una entrega aislada: cada bloque deja una evidencia concreta del ciclo.

#### **Indicaciones de demo o énfasis**

- Elegir 3 o 4 filas representativas; no leer la tabla completa.

*Transición: Una vez situado el mapa de competencias, paso a la arquitectura general.*

## **07. Arquitectura general (Fig. 1)**

**Tiempo estimado: 5:00-5:45**

*Objetivo comunicativo: Explicar fronteras entre aplicación, datos, monitorización y simulador.*

#### **Mio:**

La arquitectura la podríamos dividir en tres capas, la web que se compone de Apache y PHP, la segunda la persistencia con MySQL con la base de datos relacional y la tercera con la observabilidad que consiste de Prometheus recoge las métricas Grafana las muestra y Alertmanager gestiona las alertas. (No me convence la 7 información redundante)

#### **Texto oral sugerido**

La arquitectura separa tres áreas. La primera es la aplicación web, servida por Apache y PHP. La segunda es la persistencia, con MySQL como base de datos relacional.

La tercera es la observabilidad: Prometheus recoge métricas de la aplicación, de MySQL, del nodo y de contenedores; Grafana las muestra; Alertmanager permite convertir reglas en notificaciones.

El simulador de carga queda como un componente opcional de laboratorio. No forma parte del uso normal del taller, pero sirve para demostrar comportamiento bajo tráfico y generar evidencias durante la defensa.

#### **Indicaciones de demo o énfasis**

- Señalar en el diagrama que MySQL no se expone como servicio público.

*Transición: Con esa arquitectura en mente, enseñe ahora qué puede hacer cada perfil dentro de la aplicación.*

## **08. Aplicación web — perfiles y funciones (Tabla 2)**

**Tiempo estimado: 5:45-12:00**

*Objetivo comunicativo: Demostrar la aplicación desde el punto de vista del usuario y del administrador.*

#### **Mio:**

En la web podemos observar 4 perfiles.

- El visitante puede consultar información pública, noticias y consejos, registrarse, iniciar sesión y solicitar una citación como invitado si no tiene cuenta.

- Puede realizar las acciones anteriores mas cambiar contraseña y revisar su perfil.
  - El administrador gestiona usuarios, citas, noticias, consejos, estadísticas y accesos a monitorización. No es un rol para el taller sino que es la persona que monta la infraestructura.
  - . Y el operador técnico no es un rol de negocio de la web, sino la persona que despliega, revisa logs, comprueba healthchecks, lanza simulaciones y valida backups.
- (No me gusta el Cuarto rol, no lo entiendo)

### **Texto oral sugerido**

La aplicación distingue cuatro perfiles. El visitante puede consultar información pública, noticias y consejos, registrarse, iniciar sesión y solicitar una citación como invitado si no tiene cuenta.

El usuario registrado puede reservar citas, consultar sus citas propias, revisar su perfil y cambiar la contraseña. En el dominio del proyecto, la citación es la acción de reservar; si se confirma, se crea una cita persistida en base de datos.

El administrador gestiona usuarios, citas, noticias, consejos, estadísticas y accesos a monitorización. Y el operador técnico no es un rol de negocio de la web, sino la persona que despliega, revisa logs, comprueba healthchecks, lanza simulaciones y valida backups.

En la demo voy a recorrer el flujo principal: acceso a la aplicación, login, solicitud de cita y panel de administración. La idea es mostrar que el sistema no es una maqueta estática, sino una aplicación con roles y datos persistentes.

### **Indicaciones de demo o énfasis**

- Abrir navegador en la aplicación.
- Mostrar página pública, noticias/consejos y acceso a citaciones.
- Iniciar sesión con un usuario de prueba o administrador.
- Crear o revisar una cita sin mostrar credenciales sensibles.
- Entrar en panel admin y enseñar usuarios, citas, noticias o consejos.

*Transición: Después de ver el flujo funcional, conviene enseñar qué datos quedan detrás y como se relacionan.*

## **09. Modelo de datos y flujo de reserva (Fig. 2 y 3)**

**Tiempo estimado: 12:00-12:30**

*Objetivo comunicativo: Resumir entidades y flujo de reserva sin entrar en cada columna SQL.*

**Mio:**

Aquí hablaremos un poco del modelo de la base de datos, esta recoge todas las entidades necesarias para el taller, usuarios citas, noticias, consejos etc. Por otro lado se valida si es un usuario o un invitado, si la franja horaria esta libre y es un usuario con la sesión iniciada



permite la reserva y la guarda en el perfil, si no lo es le pedirá unos datos al invitado(Columna derecha)

### **Texto oral sugerido**

El modelo de datos recoge las entidades necesarias para el alcance del taller: usuarios, citas y contenidos como noticias o consejos.

La parte clave es que la reserva no queda como texto suelto. Se valida una fecha y una franja horaria, se registran los datos necesarios y se crea una cita vinculada a usuario o a datos de invitado.

Esto permite trazabilidad: el administrador puede ver, editar o eliminar la cita desde su panel, mientras que el usuario consulta su historial desde citaciones.

### **Indicaciones de demo o énfasis**

- No explicar cada campo; hablar de relaciones, unicidad y trazabilidad.

*Transición: Una vez visto el modelo, paso a cómo se implanta todo el stack con Docker Compose.*

## **10. Implantación con Docker Compose (Tabla 6)**

**Tiempo estimado: 12:00-18:00**

*Objetivo comunicativo: Defender reproducibilidad, redes internas, volúmenes y aislamiento de servicios.*

### **Mio:**

Docker Compose permite levantar el sistema como un conjunto de servicios coordinados, la web que contiene Apache y PHP estos esperan a que MySQL este sano antes de arrancar.

MySQL guarda la persistencia y se comunica con la aplicación por la red interna.

Prometheus y alertmanager se encargan del scripting y de las alertas.

Grafana carga los dashboards .

Y JMeter que se encarga de la carga sintética.

### **Texto oral sugerido**

Docker Compose permite levantar el sistema como un conjunto de servicios coordinados. El servicio web contiene Apache y PHP, y espera a que MySQL esté sano antes de arrancar completamente.

MySQL guarda la persistencia y, en un despliegue serio, no se publica al host. Esto reduce superficie de ataque: la aplicación habla con la base de datos por la red interna de Docker, no por un puerto abierto hacia fuera.

Prometheus y Alertmanager se encargan de scraping y alertas; Grafana carga dashboards preprovisionados desde la carpeta de monitorización; y los exportadores aportan métricas de nodo, MySQL, blackbox, cAdvisor y Telegraf.

El perfil traffic añade JMeter worker y una interfaz para pruebas controladas. Lo separo como perfil porque no es necesario para el uso diario del taller, pero si para validar carga y generar evidencias.

### **Indicaciones de demo o énfasis**

- En terminal, ejecutar o mostrar `docker compose ps`.
- Señalar servicios esperados: web, mysql, prometheus, alertmanager, grafana, exportadores y simulador si está activo.
- Explicar puertos publicados y puertos privados.
- Si se enseña `.env`, ocultar secretos o usar solo variables no sensibles.

*Transición: El simulador enlaza directamente con una de las decisiones más originales del proyecto: distinguir tráfico real y tráfico sintético.*

## **11. Originalidad: fuente de tráfico en métricas**

**Tiempo estimado: 18:00-18:30**

*Objetivo comunicativo: Explicar la decisión de etiquetar fuentes de tráfico para no mezclar evidencias.*

**Mio:**

En monitorización era importante no mezclar el tráfico real con el que generamos con Jmeter. Así que las métricas incorporan etiquetas de fuente de tráfico

### **Texto oral sugerido**

En monitorización era importante no mezclar el uso real de la aplicación con la carga generada por JMeter. Por eso las métricas HTTP incorporan una etiqueta de fuente de tráfico. El que genera la aplicación sería source=app y el sintético source=simulator. Ambos logs llegan al mismo archivo pero con una etiqueta diferente.

Las peticiones de la aplicación quedan como source=app y las importadas desde JMeter como source=simulator. Así puedo comparar comportamiento sin presentar tráfico sintético como si fuera actividad real del taller.

La decisión está documentada en los ADRs: se reutiliza un pipeline por logs y un único camino de scraping, evitando instrumentar cada ruta PHP solo para la demo.

### **Indicaciones de demo o énfasis**

- Usar el diagrama para decir que el simulador escribe en el mismo modelo de métricas, pero etiquetado.

*Transición: Con esa etiqueta, Prometheus y Grafana pueden separar claramente qué está ocurriendo.*

## **12. Monitorización y observabilidad**

**Tiempo estimado: 18:30-20:00**

*Objetivo comunicativo: Explicar el sistema de observabilidad y las métricas que se defenderán.*

**Mio:**

Prometheus recoge las métricas de la aplicación PHP mediante metrics.php, de MySQL gracias a Blackbox. Para la aplicación utilizamos métricas RED cuántas peticiones llegan, cuántas fallan y que latencia tienen. Grafana coge los datos para crear los dashboards y Alert manager queda a la espera por si debe lanzar una alerta.

**Texto oral sugerido**

Prometheus recoge métricas de la aplicación PHP mediante metrics.php, de MySQL, del sistema y de probes HTTP con blackbox. El objetivo es cubrir salud de servicio, rendimiento y datos de negocio.

Para la aplicación uso métricas RED: rate, errors y duration. Es decir, cuántas peticiones llegan, cuántas fallan y que latencia tienen. Al añadir la etiqueta source puedo ver esas métricas para tráfico real o simulado.

Grafana consume Prometheus y muestra dashboards preprovisionados. Alertmanager queda preparado para convertir reglas en notificaciones, por ejemplo alertas de caída de métricas, errores 5xx o problemas de probe HTTP.

**Indicaciones de demo o énfasis**

- Preparar la demo de Grafana: targets, dashboard principal y paneles JMeter/source.

*Transición: Ahora muestro las evidencias: contenedores vivos, targets sanos y paneles con datos.*

### **13. Evidencias: monitorización y contenedores (Fig. 22 y 20)**

**Tiempo estimado: 20:00-24:00**

*Objetivo comunicativo: Mostrar evidencias operativas en vivo sin exponer credenciales.*

**Mio:**

Creo que sobra esta diapositiva.

**Texto oral sugerido**

En esta parte no quiero limitarme a capturas. La defensa debe demostrar que el sistema está operando: contenedores levantados, Prometheus scrapeando targets y Grafana visualizando series.

Primero reviso Prometheus targets para confirmar que la aplicación, MySQL y exportadores responden. Después paso a Grafana y enseño el dashboard principal: estado general, tráfico, errores, latencia, base de datos, recursos y métricas de negocio.

Si lanzo una carga breve con JMeter, la lectura importante es que la actividad aparece como source=simulator. Eso permite demostrar carga sin confundirla con usuarios reales.

También puedo enseñar que el endpoint metrics.php centraliza métricas de aplicación y que blackbox comprueba disponibilidad HTTP desde fuera de la propia aplicación.

### Indicaciones de demo o énfasis

- Abrir Prometheus `/targets` y comprobar targets UP.
- Abrir Grafana y seleccionar el dashboard principal.
- Mostrar paneles de requests, errores, latencia, MySQL y negocio.
- Ejecutar carga breve solo si el entorno está estable y hay tiempo.
- No proyectar credenciales, tokens ni paneles de configuración sensible.

*Transición: Tras ver que el sistema funciona, explico las medidas de seguridad y las decisiones de exposición.*

## 14. Seguridad de aplicación e infraestructura

**Tiempo estimado: 24:00-25:30**

*Objetivo comunicativo: Defender mitigaciones de aplicación e infraestructura.*

### Mio:

-La seguridad se trabaja en dos niveles. En la aplicación hay sesiones PHP, validación de entrada, roles de administrador y usuario y CSRF.

-En la API de citas las contraseñas no se guardan en texto plano sino que se cifran con el mecanismo de hash de PHP e incorpora un rate limiting (evita spam).

-La red Aquí tenemos los security groups, como hemos dicho antes, JMeter, MySQL no se exponen

-El simulador también tiene token de control, límites de URL

### Texto oral sugerido

La seguridad se trabaja en dos niveles. En la aplicación hay sesiones PHP, validación de entrada, roles de administrador y usuario, CSRF en formularios críticos y consultas preparadas para reducir riesgo de SQL injection.

Las contraseñas no se guardan en texto plano: se cifran con el mecanismo de hash de PHP. En la API de citas se incorpora rate limiting para reducir spam de reservas.

En infraestructura, la decisión principal es cerrar lo que no debe ser público. MySQL no se expone al host; Prometheus, Grafana, Alertmanager y la UI del simulador deben quedar restringidos por Security Groups o por rutas controladas.

El simulador también tiene token de control, límites de URL y recomendación de TLS si se expone la interfaz.

### Indicaciones de demo o énfasis

- Preparar respuesta a la pregunta: por qué MySQL no está expuesto al host.

*Transición: Estas medidas se trasladan al despliegue cloud documentado en AWS.*

## 15. Despliegue en AWS (EC2 + Docker Compose)

**Tiempo estimado: 25:30-26:30**

*Objetivo comunicativo: Presentar una arquitectura cloud realista para el alcance del PFC.*

**Mio:**

Para este proyecto se ha decidido utilizar una instancia EC2 y Docker Compose, por la complejidad y motivo económico.

La guía contempla una AMI base con Docker mediante Packer, o un user-data que instala Docker , clona el repositorio en /opt/taller\_mecanico\_asir y ejecuta el script de despliegue.

Se rotan los secretos de .env

### **Texto oral sugerido**

El despliegue en AWS se plantea con una instancia EC2 y Docker Compose. Para este tamaño de proyecto es una solución razonable: sencilla, económica y suficiente para demostrar operación.

La guía contempla una AMI base con Docker mediante Packer, o un user-data idempotente que instala Docker, clona el repositorio en `/opt/taller\_mecanico\_asir` y ejecuta el script de despliegue.

El disco EBS debe ir cifrado, los secretos se rotan en `.env`, y las reglas de entrada se limitan a HTTP/HTTPS y, si hace falta en laboratorio, a las UIs de monitorización desde una IP concreta.

Para una fase posterior, una mejora natural sería mover MySQL a RDS y automatizar backups en S3.

### **Indicaciones de demo o énfasis**

- Si se abre consola EC2, mostrar solo arquitectura o estado; evitar enseñar secretos o claves.

*Transición: Después de arquitectura, queda demostrar como se ha validado el proyecto.*

## 16. Validación y pruebas

**Tiempo estimado: 26:30-27:00**

*Objetivo comunicativo: Resumir pruebas funcionales, despliegue y trazabilidad requisito-evidencia.*

**Mio:**

(Eliminar esta diapositiva)

### **Texto oral sugerido**

La validación se organiza en tres grupos. Primero, pruebas funcionales: registro, login, citaciones, noticias, consejos y administración.

Segundo, pruebas de despliegue y monitorización: arranque Docker, healthchecks, targets de Prometheus y dashboards de Grafana.

Tercero, pruebas con simulador: carga breve y diferenciación entre source=app y source=simulator. La matriz de pruebas enlaza requisito con evidencia, que es lo importante para defender el grado de consecución.

#### **Indicaciones de demo o énfasis**

- Mantener esta diapositiva breve; el detalle ya se habrá visto en las demos.

*Transición: Con las pruebas cerradas, presento el coste orientativo de operar esta solución.*

### **17. Presupuesto anual orientativo (Tabla 14)**

**Tiempo estimado: 27:00-27:30**

*Objetivo comunicativo: Dar una cifra memorable y prudente de coste anual.*

**Mio:**

#### **Texto oral sugerido**

El presupuesto anual orientativo ronda los 1.146 euros al año. Incluye una EC2 t3.small funcionando 24/7, disco EBS cifrado, dominio, backups o snapshots, correo SMTP para alertas, implantación, mantenimiento y una contingencia del 10 por ciento.

Es una estimación para tener orden de magnitud. En producción real dependería de región, tráfico, dominio, política de backups y si se incorpora un balanceador, RDS u otros servicios gestionados.

#### **Indicaciones de demo o énfasis**

- No discutir céntimos; defender el coste como estimación de explotación.

*Transición: Con coste y alcance claros, cierro con conclusiones y mejoras.*

### **18. Conclusiones y mejoras futuras**

**Tiempo estimado: 27:30-28:00**

*Objetivo comunicativo: Cerrar con balance honesto: conseguido, límites y futuro.*

**Mio:**

Como conclusión, el proyecto alcanza los objetivos principales: aplicación web operativa, base de datos relacional, despliegue reproducible local y cloud, y monitorización activa.

También es importante reconocer límites, queda margen de mejora en la seguridad de la web, tener backups en S3...

#### **Texto oral sugerido**

Como conclusión, el proyecto alcanza los objetivos principales: aplicación web operativa, base de datos relacional, despliegue reproducible local y cloud, y monitorización activa.

También es importante reconocer límites: queda margen para endurecimiento adicional, TLS definitivo, automatización de backups externos y políticas RGPD más completas.

Las mejoras naturales serían RDS para base de datos gestionada, CI/CD, backups en S3 y una política formal de continuidad.

#### **Indicaciones de demo o énfasis**

- Usar tono honesto: conseguido no significa cerrado para producción exigente.

*Transición: La última tabla resume el grado de consecución de cada objetivo.*

## **19. Demostración, preguntas y cierre**

**Tiempo estimado: 28:30-30:00+**

*Objetivo comunicativo: Cerrar, agradecer y abrir preguntas con una posición técnica clara.*

**Mio:**

Muchas gracias por la atención y al tutor por el seguimiento del proyecto Gracias.

#### **Texto oral sugerido**

Para cerrar, el proyecto demuestra un recorrido completo: aplicación, datos, despliegue, observabilidad, seguridad básica, coste y validación.

Si queda tiempo, puedo repetir una comprobación rápida de operador técnico: aplicación, `docker compose ps`, Prometheus/Grafana y una carga breve. Si no, prefiero dejar margen para preguntas del tribunal.

Muchas gracias por la atención y al tutor por el seguimiento del proyecto. Quedo a disposición para preguntas.

#### **Indicaciones de demo o énfasis**

- Checklist opcional: app -> docker compose ps -> Grafana/Prometheus -> carga breve.
- Priorizar preguntas frente a demos adicionales si el tiempo está justo.

*Transición: Fin de exposición.*

## **Preguntas probables del tribunal**

### **¿Por qué no expones MySQL al host o a Internet?**

Porque MySQL solo debe ser consumido por la aplicación dentro de la red Docker. Publicarlo aumentaría la superficie de ataque sin aportar valor al usuario final. Para administración se usarían canales controlados: shell, backup, túnel o herramientas internas.

### **¿Qué diferencia hay entre source=app y source=simulator?**

source=app representa tráfico real generado por la aplicación. source=simulator representa peticiones importadas desde JMeter. La separación evita confundir carga sintética con uso real y permite comparar latencia, errores y volumen por origen.

### **¿Por qué usar logs para métricas HTTP y no instrumentar cada ruta PHP?**

El pipeline por logs permite un único camino de observabilidad para tráfico real y simulado, reduce cambios en rutas PHP y encaja con un enfoque ASIR centrado en operación. La desventaja es que exige formato de log consistente y control de volumen.

### **¿Qué aporta Prometheus/Grafana si la aplicación ya funciona?**

Aporta visibilidad operativa. Permite saber si la aplicación responde, si hay errores 5xx, cómo evoluciona la latencia, si MySQL tiene problemas y si los contenedores consumen demasiados recursos. Sin observabilidad solo se descubre el fallo cuando el usuario se queja.

### **¿Por qué Docker Compose en vez de Kubernetes?**

Para el alcance del PFC y una PYME pequeña, Compose es más proporcionado: menos complejidad, despliegue reproducible y suficiente para demostrar servicios, redes, volúmenes y monitorización. Kubernetes sería justificable con escalado, alta disponibilidad o varios equipos.

### **¿Qué faltaría para producción real?**

TLS definitivo, dominio real, políticas RGPD revisadas, backups externos probados, restauración documentada, rotación formal de secretos, endurecimiento de UIs de monitorización y posiblemente RDS para reducir operación de base de datos.

### **¿Cómo validas que una cita no se duplica?**

El flujo valida fecha y franja antes de crear el registro y el modelo relacional impone coherencia. En la defensa conviene explicar que la aplicación no trata la cita como texto libre, sino como registro persistido y administrable.



## **¿Qué pasa si Grafana no muestra datos durante la defensa?**

Hay un plan de contingencia: revisar Prometheus targets, comprobar `metrics.php`, verificar contenedores con `docker compose ps` y usar capturas/evidencias incluidas en la presentación si el entorno de demo falla.

## **Cierre de ensayo**

Si hay desviación de tiempo, recortar antes las tablas y conservar las evidencias: app, Docker, Grafana y separación `source=app/source=simulator`. La defensa debe transmitir criterio operativo, no solo enumerar herramientas.